

DrapixAI

Project dossier covering product, technology, deployment, launch readiness, and go-to-market context.

Date: 2026-06-26

Audience: founder, sales, engineering, launch, and support

Current scope: upper-body AI try-on, standard quality only

1. Executive Summary

DrapixAI is a B2B AI virtual try-on platform for fashion ecommerce. It is being built as a controlled rollout product rather than a broad, hands-off consumer AI tool.

The current product focus is upper-body garments only. The system emphasizes garment validation, confirmed product mapping, cached garment assets, quality controls, and operational readiness before scale.

2. What The Product Does

- Lets a brand create an account and obtain an API key.
- Accepts isolated garment images and validates them before use.
- Preprocesses garments into cached assets for repeatable try-on quality.
- Syncs product catalog data and suggests garment-to-product matches.
- Requires manual confirmation of final product mappings before storefront traffic uses them.
- Generates shopper try-on outputs from a shopper photo plus confirmed product identifier.
- Returns response metadata such as quality score, latency, warnings, engine, and cache state.

3. Current Launch Scope

Launch-ready categories are focused on upper-body garments. Stronger categories include shirts, t-shirts, polos, blouses, and clean tops. Short kurtis, hoodies, and sweatshirts are more cautious categories.

The product should not currently promise full-body support, every apparel class, or a universal cross-platform rollout. Full-body is positioned as future work rather than part of the current public launch promise.

4. Customer Workflow

- Create account and log in.
- Receive or rotate API key.
- Upload garment-only images.
- Run garment validation and cache generation.
- Sync catalog products into the dashboard.
- Review suggested matches.
- Confirm the correct garment-to-product pairings manually.
- Install the browser SDK or call the REST API.
- Send shopper photo plus confirmed product context.
- Receive try-on image and metadata.

5. Product Features

- Authentication: email/password signup, OTP verification, login, optional Google OAuth.
- Dashboard: usage visibility, analytics, account settings, and plan visibility.
- API key controls: active key lifecycle, rotation, and domain-lock behavior.
- Garment pipeline: upload, validation, preprocessing, cache generation, thumbnails, and approval states.
- Catalog flow: catalog sync, discovered products, suggested matches, and confirmed mapping.
- Try-on flow: SDK and REST API for standard-quality upper-body generation.
- Operations: readiness endpoints, admin review flows, support contacts, and email logging.

6. Pricing And Commercial Shape

The commercial motion is designed to be low-risk: brands validate the workflow on their own garments first, then move into paid usage only when the try-on flow is trustworthy enough for live rollout.

One practical note: pricing language and some backend quota values should be reconciled before billing is treated as final.

- Trial: 300 try-ons over 12 days.

- Starter: entry paid plan for smaller brands.
- Growth: higher-volume plan for stores with more usage.
- Pro: shown as future-facing / coming soon.
- Enterprise: sales-led custom path.

7. Web Stack

- Framework: Next.js 16.
- Frontend runtime: React 18.
- Styling: Tailwind CSS.
- Session/auth integration: NextAuth.
- Public pages include homepage, pricing, help, contact, auth flows, dashboard, admin, and settings-related views.

8. Backend Stack

The API is organized around auth routes, SDK routes, analytics routes, admin routes, account routes, and public routes. It also exposes health and ready checks for operations.

- Runtime: Node.js with Express and TypeScript.
- ORM and schema layer: Prisma.
- Primary data store: PostgreSQL.
- Queue/cache coordination: Redis.
- Email delivery: Nodemailer via SMTP.
- Scheduled background work: node-cron.

9. AI Stack

- Runtime: Python FastAPI service.
- Core try-on engine: CatVTON pipeline.
- Garment pipeline: preprocessing, validation, cache storage, and retrieval.
- Request safety: internal service token for API-to-AI calls.
- Upper-body enforcement and standard-quality enforcement in the live path.
- Response metadata includes quality score, timings, warnings, candidate count, and garment source.

10. Deployment Topology

The repo already contains Dockerfiles, environment templates, nginx config, RunPod bootstrap scripts, environment validation scripts, smoke tests, and health-check helpers.

- Edge host: public web app, API, and nginx reverse proxy.
- GPU host: RunPod A100 pod running AI API and worker.
- Managed Postgres recommended for production.
- Managed Redis recommended for production.
- S3-compatible object storage used for uploads, outputs, and thumbnails.
- SMTP provider used for OTP and transactional email.
- Optional Google OAuth for sign-in.

11. Recommended AI Runtime

Project docs explicitly say that Windows or local output should not be treated as the final production quality gate. The production quality gate is the Linux RunPod GPU path.

- Preferred live environment: RunPod Ubuntu GPU stack.
- Preferred GPU: A100 PCIe 80GB.
- Base image: runpod/pytorch 2.4.0 with Python 3.11 and CUDA 12.4.1.
- Preset path: DRPIXAI_GPU_PRESET=runpod-a100.
- Warm latency target: about 10 to 12 seconds.
- Model preload is enabled on the A100 path to reduce cold-start impact.

12. Core Data Model

- User: account, company, plan, trial, store verification, and catalog sync metadata.
- ApiKey: active key storage and domain whitelist.
- Usage and UsageDaily: monthly and daily consumption tracking.
- Garment: garment assets, cache keys, thumbnails, status, and source references.
- CatalogProduct: discovered or synced product catalog items.
- GarmentMatch: suggested and confirmed mapping between garment and product.
- TryOnResult: generated result image, quality score, timing, warnings, and approval status.
- TryOnFeedback: manual realism or defect review data.
- EmailLog: OTP and transactional email tracking.
- VerificationCode: OTP and email verification support.

13. Security And Operational Controls

- Helmet and CORS protections on the API.
- JWT-based account auth.
- Bcrypt hashing for passwords and API keys.
- Single-domain enforcement behavior for API keys in the live SDK path.
- Readiness checks across database, Redis, AI, and storage.
- Admin review filters for low quality, high latency, warnings, and cache issues.
- Email send logging for supportability.

14. Marketing And Go-To-Market Assets

The repository already contains a substantial founder-led marketing pack. That means the business side of launch has been prepared alongside the product and infrastructure.

- Go-to-market strategy document.
- Video scripts and production kit.
- Prelaunch partner brief.
- Cold outbound email sequence and sender setup guide.
- Growth experiments and omnichannel roadmap.
- Segmented email templates for growth and enterprise accounts.
- Lead templates and a marketing-agent operating framework.

15. Launch Readiness

The codebase appears structurally close to launch. Deployment documentation, environment templates, health checks, smoke-test scripts, AI presets, and support playbooks are already in place.

However, the project docs still call out a few live blockers that matter before public launch.

- Live RunPod A100 validation.
- Real SMTP verification.
- Optional Google OAuth verification if exposed publicly.
- At least one successful end-to-end public try-on on the Linux GPU path.

16. Practical Assessment

DrapixAI should be understood as a controlled AI commerce system, not just an image-generation feature. Its real value comes from combining garment preparation, product mapping, AI generation, analytics, and launch operations into one commerce workflow.

From the repository state, the project already shows serious product, engineering, and go-to-market preparation. The main remaining work is live production proof on the target A100 stack and finalizing the external infrastructure pieces needed for a safe public launch.

17. Source Snapshot Used For This Brief

- deploy.md
- deploy/production-readiness.md
- deploy/launch-support-playbook.md
- apps/web/app/page.tsx
- apps/web/app/pricing/page.tsx
- apps/web/app/help/page.tsx
- apps/web/app/contact/page.tsx
- apps/api/prisma/schema.prisma
- apps/api/src/server.ts
- apps/api/src/routes/auth.ts
- apps/api/src/routes/sdk.ts
- apps/api/src/routes/analytics.ts
- apps/api/src/services/emailer.ts
- drapixai_ai/api/ai_server.py
- drapixai_ai/configs/settings.py
- marketing/README.md